Project: To-Do List API **CR ID:** CR-003 — Scheduled Overdue Tasks

---

1. Technical Objective

Automatically identify tasks that are overdue (`dueDate` in the past, `status != DONE`) on a recurring schedule, using Spring's built-in scheduling support, without introducing a messaging system or changing the existing `Task` status model.

2. Proposed Solution

Add a `@Scheduled` component that, on a configurable cron trigger, calls a repository query for overdue tasks and logs the result. The detection logic is implemented as a plain, directly-callable service method so it can be unit tested without depending on the scheduler actually firing (FR-05). The scheduler bean itself is a thin wrapper that simply invokes that method on its configured trigger.

3. Components to Modify

Component	File	Change
Main application class	`TodoApiApplication.java`	Add `@EnableScheduling`
`application.yml`	config	Add a configurable cron expression property
Task repository	`repository/TaskRepository.java`	Add an overdue-tasks query method

New components:

Component	Purpose
`scheduler/OverdueTaskScheduler.java`	`@Component` holding the `@Scheduled` method; delegates to the service below
`service/OverdueTaskService.java` (+ impl)	Directly-callable, testable detection logic (FR-05)
`entity/TaskNotification.java` *(only if Option B is approved — see §9)*	Persisted notification record
`repository/TaskNotificationRepository.java` *(Option B only)*	Standard `JpaRepository` , plus an `existsByTaskId` check for FR-06/FR-07

4. Database Changes (JPA/Hibernate)

Option A (detect and log only): no schema change at all — the job reads `Task` via a new repository query; nothing new is persisted.

Option B (detect + non-duplicated notifications), if approved:

```
@Entity
@Table(name = "TASK_NOTIFICATION")
public class TaskNotification {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "TASK_ID", nullable = false)
    private Task task;

    @Column(name = "DETECTED_AT", nullable = false)
```

```
        private Instant detectedAt;

        @Column(name = "DUE_DATE_AT_DETECTION", nullable = false)
        private LocalDate dueDateAtDetection;
    }
```

- A **unique constraint on** `(TASK_ID, DUE_DATE_AT_DETECTION)` prevents a duplicate notification for the same task against the same due date across multiple scheduler runs (FR-06/FR-07). If the task's due date is later changed and it becomes overdue again under a new due date, a new notification is allowed, since it reflects a genuinely new state.
- No migration risk — new table, no existing data affected.

Repository query (both options)

```
    @Query("SELECT t FROM Task t WHERE t.dueDate < :today AND t.status <>
com.example.todoapi.enums.TaskStatus.DONE")
    List<Task> findOverdueTasks(@Param("today") LocalDate today);
```

Passing `today` as a parameter (rather than using a database function) keeps the "current business date" concept testable and explicit (FR-05), and sidesteps timezone ambiguity between the application server and the Oracle instance (see §6).

5. API Design

Option A: no API change — this is a background job only.

Option B: adds one read-only endpoint, consistent with the project's existing REST conventions:

```
GET /api/tasks/overdue
```

Response `200 OK`:

```
    [
      {
        "taskId": 17,
        "title": "Submit vendor contract",
        "dueDate": "2026-10-10",
        "detectedAt": "2026-10-15T06:00:00Z"
      }
    ]
```

This endpoint is additive only — it does not change any existing endpoint's contract.

6. Processing Logic

1. `OverdueTaskScheduler.run()` is invoked by Spring's scheduler on the configured cron trigger (e.g. daily at 06:00 server time, via `app.scheduler.overdue-cron` in `application.yml`).
2. It delegates to `OverdueTaskService.detectOverdueTasks(LocalDate.now())` — passing the current date explicitly, so the same method can be called from a test with an arbitrary date (FR-05).
3. The service calls `taskRepository.findOverdueTasks(today)`.
4. For each task returned:
   - **Option A:** log `taskId`, `title`, `dueDate` at `INFO` level and accumulate a count.
   - **Option B:** additionally check `taskNotificationRepository .existsByTaskIdAndDueDateAtDetection(taskId, task.getDueDate())`; if not already present, insert a new `TaskNotification` row.
5. After processing all matches, log a single summary line: total overdue tasks found, and (Option B) how many new notifications were created versus already existing.
6. **Timezone handling:** the job uses `LocalDate.now()` from the application server's configured zone (to be made explicit in `application.yml` rather than relying on JVM default), compared against `Task.dueDate` (already a `LocalDate`, timezone-naive) — avoiding any timestamp/timezone conversion issue, since both sides of the comparison are calendar dates, not instants.

7. Error Handling

Scenario	Behavior
Scheduler run throws an unexpected exception (e.g. transient DB issue)	Exception is caught and logged at `ERROR` level inside the scheduled method; it must not propagate and must not crash the JVM or deregister the schedule — Spring's `@Scheduled` does not retry automatically
Query returns zero overdue tasks	Logged as a normal, successful run with count `0` — not an error
(Option B) Duplicate notification attempted for an unchanged due date	Prevented by the unique constraint; a constraint violation on this specific insert is treated as "already notified," not surfaced as an error

8. Test Scenarios

See `TEST_CASES.md`. Summary: detection correctly separates overdue from non-overdue tasks (including the today-is-not-overdue and DONE-is-never-overdue edge cases); the service method is callable directly without the scheduler; repeated runs don't re-flag the same task (Option B); the cron expression is externally configurable.

9. Technical Decision

Selected approach: Spring `@Scheduled` with an externalized cron expression, delegating to a plain, directly-testable service method.

Reason for selection: Uses only what Spring Boot already provides (no new dependency), keeps detection logic decoupled from the scheduling trigger (satisfies FR-05 directly), and fits the project's existing Controller→Service→Repository layering (the scheduler is just another caller of a service method, the same way a controller is).

Alternatives considered:

- **A separate scheduling library (e.g. Quartz):** rejected — unnecessary for a single, simple recurring job; Spring's built-in scheduler is sufficient and avoids adding a dependency per the project's "minimal implementation complexity" architecture rule.
- **Detecting overdue status lazily on every `GET` request instead of a background job:** rejected — the CR explicitly asks for scheduled, proactive detection, and computing this on every read would also make the "notification, created once" requirement (FR-06/FR-07) impossible to satisfy cleanly.

Recommended scope for Option A vs. B: Option A (detect and log only) is recommended as the initial, demo-appropriate scope. It satisfies the CR's core business problem (overdue tasks are not currently detected) with the least new surface area — no new table, no new endpoint, no duplicate- prevention logic to get right. Option B is a reasonable next increment once Option A is validated, but is explicitly a **proposed scope decision for the project owner to confirm, not an approved requirement** — it assumes a consumer for notification records that does not yet exist (there is no notification-delivery system or authenticated user to notify in this project).

10. Technical Approval

- **Prepared By:** Backend Developer
- **Reviewed By:** Tech Lead
- **Approval Status:** Pending
- **Implementation Status:** Not Started